Microsoft

AI-200T00A

Develop AI solutions with Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

Agenda

- Build queries for Azure Cosmos DB for NoSQL
- Implement vector search with Azure Cosmos DB for NoSQL
- Optimize query performance for Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

Module 1: Build queries for Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain the Azure Cosmos DB for NoSQL resource model and how databases, containers, and items relate to each other
- Implement SDK operations to connect to Azure Cosmos DB and perform CRUD operations on items
- Select between point reads and queries based on performance requirements and access patterns
- Build queries using SQL syntax to filter, project, and retrieve data from containers

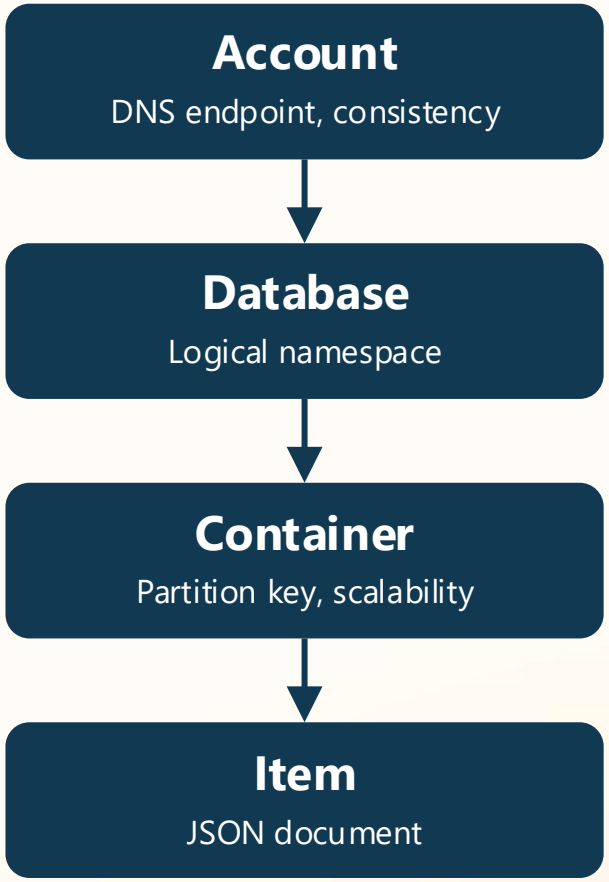
© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Globally distributed, multi-model database service with guaranteed low latency
- NoSQL API supports flexible JSON document storage with no fixed schema
- Automatic indexing on all properties by default
- Throughput measured in Request Units (RUs) with manual or autoscale options
- Multiple consistency levels from strong to eventual
- Native SDK support for Python, .NET, Java, and JavaScript

© Copyright Microsoft Corporation. All rights reserved.

Explore Azure Cosmos DB for NoSQL



Resource hierarchy

- Account → Database → Container → Item

Partition key

- Most critical design decision
- Choose high-cardinality fields: `userId`, `tenantId`, `categoryId`

Throughput

- Manual: fixed RU/s
- Autoscale: scales 10%–100% of max
- All operations measured in Request Units (RUs). Point read ~1 RU, write ~5–10 RUs

© Copyright Microsoft Corporation. All rights reserved.

Implement the Azure Cosmos DB for NoSQL SDK

Singleton client pattern

- Reuse one `CosmosClient` for the app lifecycle
- Client maintains connection pools, caches routing
- `get_database_client()` and `get_container_client()` are lightweight handles
- `create_*_if_not_exists` for idempotent setup

	Point read	Query
When	Know `id` + partition key	Need to filter/search
Cost	~1 RU (1 KB)	3–hundreds of RUs
Latency	Lowest	Varies
Method	`read_item()`	`query_items()`

© Copyright Microsoft Corporation. All rights reserved.

Query Azure Cosmos DB for NoSQL (1 of 2)

SQL syntax and filtering

```
SELECT p.id, p.name, p.price
FROM products p
WHERE p.categoryId = "electronics"
  AND p.price < 500

-- String functions
SELECT * FROM products p
WHERE CONTAINS(p.name, "Speaker")
```

Parameterized queries

```
query = """
    SELECT * FROM products p
    WHERE p.categoryId = @category
      AND p.price < @maxPrice
"""
parameters = [
    {"name": "@category",
     "value": "electronics"},
    {"name": "@maxPrice",
     "value": 500.00}
]
items = container.query_items(
    query=query,
    parameters=parameters,
    partition_key="electronics"
)
```

© Copyright Microsoft Corporation. All rights reserved.

Query Azure Cosmos DB for NoSQL (2 of 2)

Single-partition (preferred)

```
items = container.query_items(
    query=query,
    parameters=params,
    partition_key="electronics"
)
```

- Routes to one partition
- Lower RU cost, lower latency
- Specify `partition_key` param or include in `WHERE`

Cross-partition

```
items = container.query_items(
    query=query,
    parameters=params,
    enable_cross_partition_query=True
)
```

- Fans out to all partitions
- Higher RU cost
- Needed when filter doesn't include partition key

© Copyright Microsoft Corporation. All rights reserved.

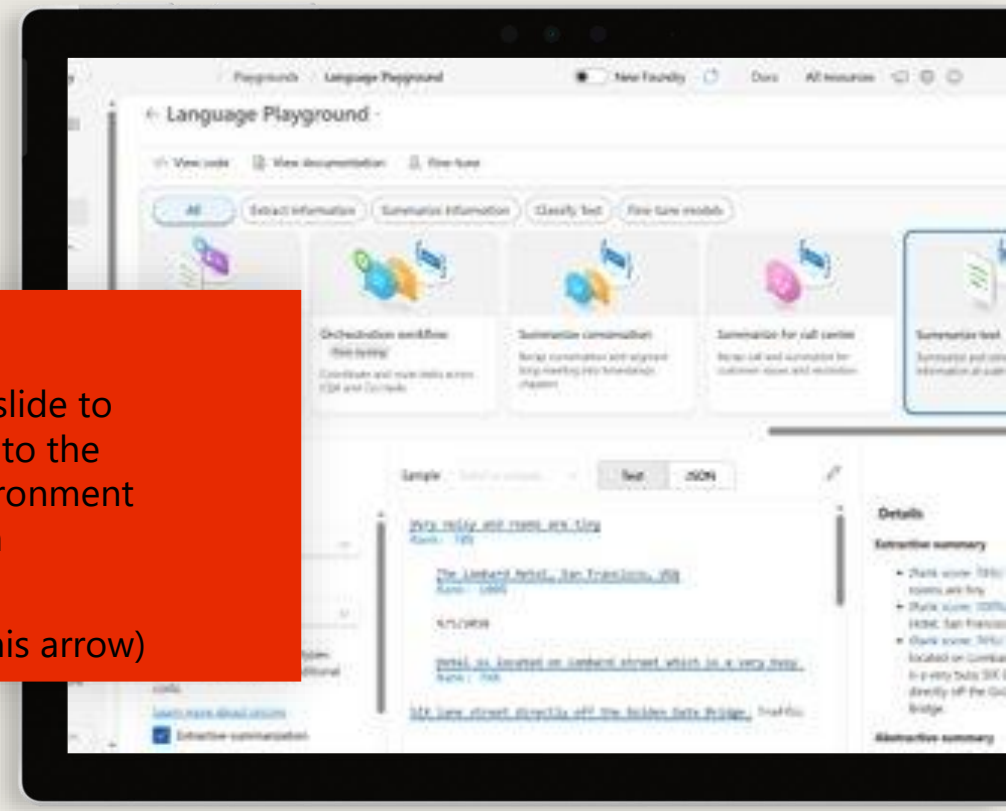
Exercise

Build a RAG document store on Azure Cosmos DB for NoSQL

In this exercise, you create an Azure Cosmos DB for NoSQL database that serves as a document store for retrieval-augmented generation (RAG) applications.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2350700



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 1 knowledge check

1

A developer is designing a container for an AI application that stores user interaction logs. Each document includes a userId property. The application frequently retrieves all logs for a specific user. Which partition key provides the best performance?

A. Use `userId` as the partition key
B. Use a timestamp property as the partition key
C. Use a boolean `isProcessed` property as the partition key

2

An AI application stores product recommendations with document IDs. The application needs to retrieve a specific recommendation by its known ID and category. Which method provides the most efficient retrieval?

A. Use `query_items()` with a `WHERE` clause filtering by `ID`
B. Use `read_item()` with the item `ID` and partition key
C. Use `query_items()` with `enable_cross_partition_query=True`

© Copyright Microsoft Corporation. All rights reserved.

Module 1 summary

- **Partition key** is your most important design decision, can’t change it later
- **Singleton client** — reuse `CosmosClient` for connection pooling and caching
- **Point reads over queries** when you know `id` + partition key
- **Parameterize queries** for security and query plan caching
- **Single-partition queries** cost less than cross-partition
- **Monitor RU consumption** to identify optimization opportunities

© Copyright Microsoft Corporation. All rights reserved.

Module 2: Implement vector search on Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Store and retrieve vector embeddings in containers with properly configured vector policies
- Execute vector similarity queries using the `VectorDistance` function
- Combine vector search with metadata filters and full-text search using hybrid queries
- Implement change feed processing to automatically refresh embeddings when documents change

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Integrated vector search in Azure Cosmos DB for NoSQL, no separate vector database required
- Store embeddings alongside business data in the same documents
- Supports multiple distance functions: cosine, dotproduct, euclidean
- Three index types: flat (exact), quantizedFlat (compressed), DiskANN (approximate)
- Hybrid search combines vector similarity with keyword and metadata filters

© Copyright Microsoft Corporation. All rights reserved.

Store and retrieve embeddings in Azure Cosmos DB

Vector policy (set at container creation)

- **path:** JSON path to embedding property
- **dataType:** `float32`, `float16`, `int8`, `uint8`
- **dimensions:** must match your model (1536 for ada-002)
- **distanceFunction:** cosine, dotproduct, euclidean

Index types

- **flat:** exact search, max 505 dimensions
- **quantizedFlat:** compressed, up to ~50K vectors/partition
- **diskANN:** best for large scale, up to 4096 dimensions

Document structure

```
{
  "id": "doc-123",
  "title": "WiFi troubleshooting",
  "content": "Common WiFi issues...",
  "category": "networking",
  "embedding": [0.023, -0.041, ...]
 }
```

Key decisions

- Exclude `/embedding/*` from standard range indexes
- Vector policies can't be modified after creation
- `float16` saves 50% storage with minimal quality loss

© Copyright Microsoft Corporation. All rights reserved.

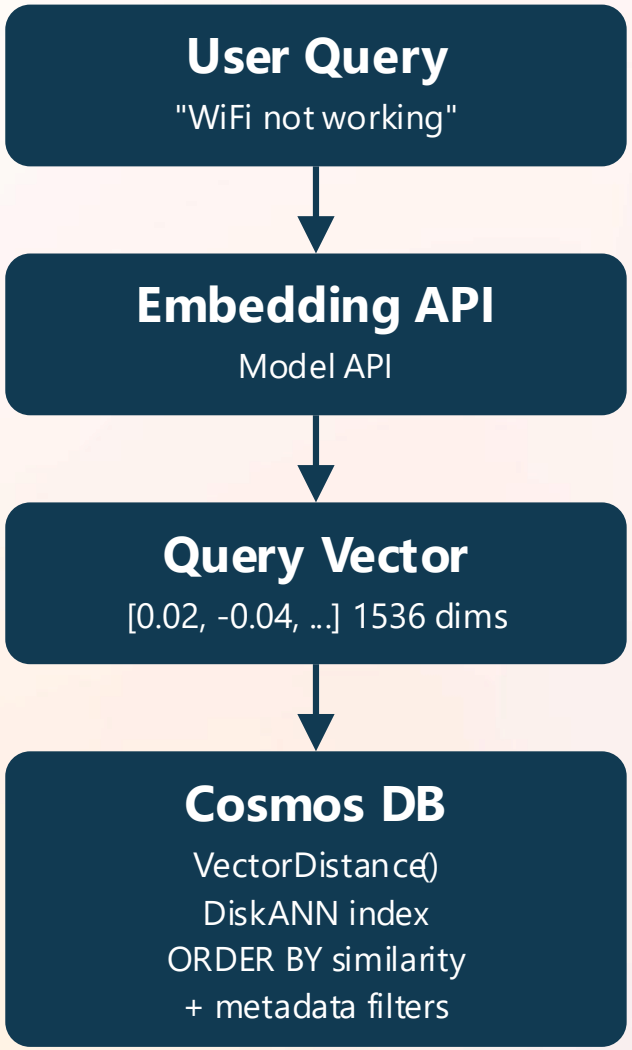
Execute vector similarity queries for semantic search

Query pattern

```
SELECT TOP 10
    c.title,
    VectorDistance(c.embedding,
        @queryVector) AS Score
FROM c
ORDER BY VectorDistance(
    c.embedding, @queryVector)
```

Essentials

- Same embedding model for queries and documents
- `TOP N` is required. Omitting it scans everything
- Parameterize with `@queryVector` for plan caching
- Scores: +1 identical, 0.7-0.9 highly similar



© Copyright Microsoft Corporation. All rights reserved.

Combine vector similarity results with metadata filtering

Filtered vector search

```
SELECT TOP 10 c.title,
    VectorDistance(c.embedding,
        @queryVector) AS Score
FROM c
WHERE c.category = @category
    AND c.createdDate > @startDate
ORDER BY VectorDistance(
    c.embedding, @queryVector)
```

- Pre-filtering reduces search space
- Partition key in WHERE enables single-partition routing
- Pass `partition_key` parameter for maximum efficiency

Hybrid search with RRF

```
SELECT TOP 10 *
FROM c
ORDER BY RANK RRF(
    VectorDistance(c.embedding,
        @queryVector),
    FullTextScore(c.content,
        @searchTerms)
)
```

- RRF merges semantic + keyword rankings
- Requires full-text policy and full-text index
- Weights adjust influence: `[2, 1]` favors vector
- Ideal for error codes, product names + natural language

© Copyright Microsoft Corporation. All rights reserved.

Use the change feed to trigger embedding refresh

Push model (Azure Functions)

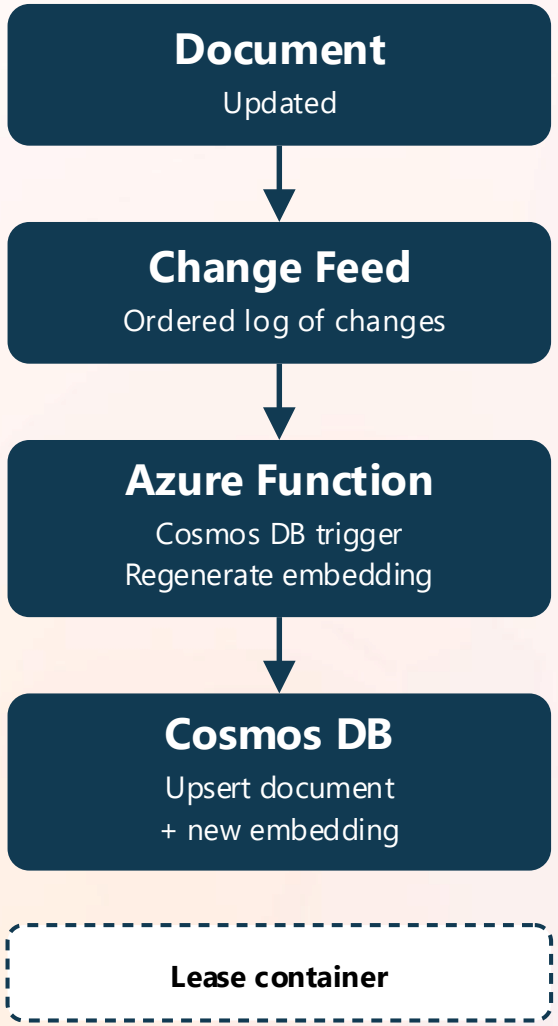
- Cosmos DB trigger auto-delivers changes
- Lease container tracks partitions and checkpoints
- Handles failover and horizontal scaling

Selective refresh

- Hash content fields to detect meaningful changes
- Metadata-only updates skip embedding regeneration
- Batch embedding API calls for high-volume scenarios

Error handling

- Processing must be idempotent
- Handle deleted documents gracefully
- Use dead-letter queues for persistent failures



© Copyright Microsoft Corporation. All rights reserved.

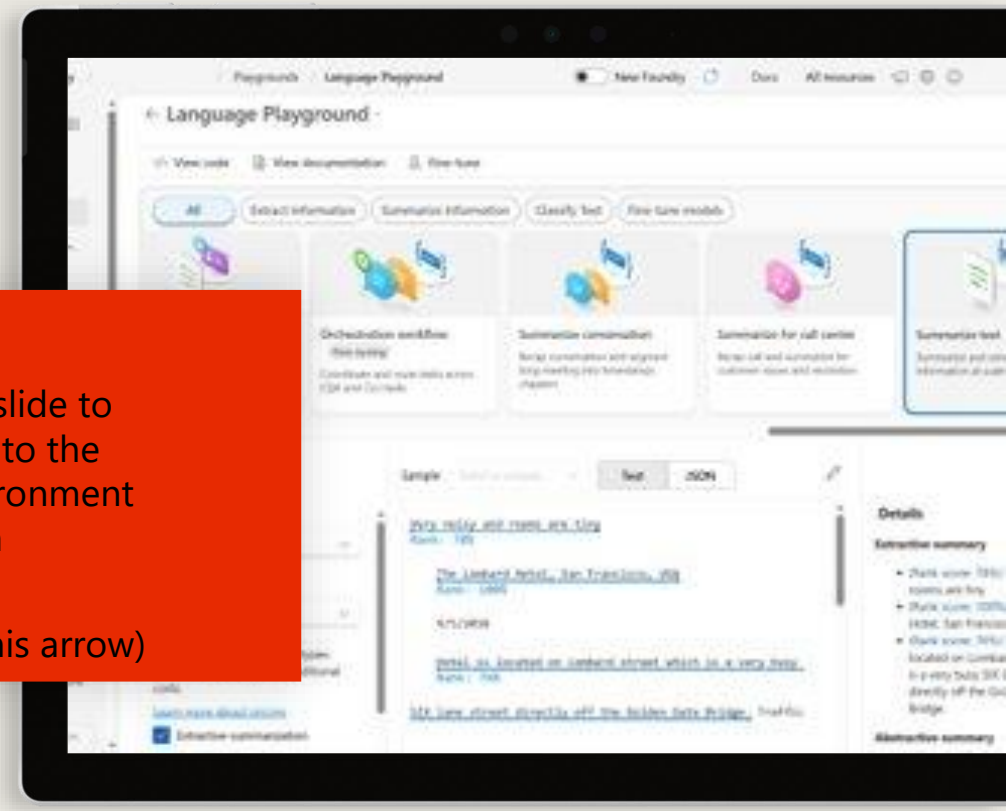
Exercise

Build a semantic search application with Azure Cosmos DB for NoSQL

In this exercise, you implement vector similarity search using Azure Cosmos DB for NoSQL.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2350601



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 2 knowledge check

1 A developer creates a container for documents with embeddings from the text-embedding-ada-002 model. Which vector policy configuration is correct for this model?

A. Set dataType to float32, dimensions to 1536, and distanceFunction to cosine
B. Set dataType to int8, dimensions to 1024, and distanceFunction to euclidean
C. Set dataType to float16, dimensions to 3072, and distanceFunction to dotproduct

2 A team wants search that combines semantic understanding with exact keyword matching for error codes. Which approach enables hybrid search?

A. Use ORDER BY RANK RRF with VectorDistance and FullTextScore functions
B. Execute two separate queries and merge results in application code
C. Add keyword terms to the query vector before executing vector search

© Copyright Microsoft Corporation. All rights reserved.

Module 2 summary

- **Vector policy** defines embedding path, data type, dimensions, and distance function at container creation
- **DiskANN index** provides fast approximate search for large-scale production workloads
- **VectorDistance + TOP N + ORDER BY** is the core query pattern for similarity search
- **Metadata filters** combine with vector search to scope results by category, date, or permissions
- **Hybrid search with RRF** merges semantic and keyword rankings in a single query
- **Change feed** with Azure Functions keeps embeddings synchronized as documents change

© Copyright Microsoft Corporation. All rights reserved.

Module 3: Optimize query performance for Azure Cosmos DB for NoSQL

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Analyze query patterns and use metrics to identify performance bottlenecks and missing indexes
- Configure range and composite indexes to optimize filter and sort operations for AI retrieval patterns
- Select the appropriate vector index type based on dataset size, accuracy requirements, and performance goals
- Design indexing policies that balance read performance against write costs for your workload profile
- Choose consistency levels that meet application requirements while minimizing RU consumption

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Cosmos DB automatically indexes all properties by default. Good for getting started, expensive at scale
- Customizing indexing policies unlocks significant performance and cost optimizations
- AI workloads have predictable access patterns: filter by metadata, sort by relevance, search by embedding
- Embedding arrays waste storage when included in range indexes. Exclude them and use vector indexes instead
- Consistency levels control the trade-off between data freshness, latency, and RU cost
- Session consistency suits most AI applications. Strong and eventual serve specialized scenarios

© Copyright Microsoft Corporation. All rights reserved.

Understand indexes in Azure Cosmos DB

Index types

- Range: equality, range filters, single-property ORDER BY
- Composite: multi-property sort and combined filter + sort
- Vector: VectorDistance similarity search (flat, quantizedFlat, diskANN)
- Spatial: geospatial queries (ST_DISTANCE, ST_WITHIN)

Path syntax

- `/*` indexes all properties recursively
- `/property/?` indexes a scalar value
- `/array/[]` indexes array elements
- More specific paths override general ones

© Copyright Microsoft Corporation. All rights reserved.

Configure range and composite indexes

Range indexes for filters

```
SELECT * FROM c
WHERE c.documentType = 'pdf'
  AND c.uploadDate > '2024-01-01'
```

Composite indexes for sorting

```
SELECT * FROM c
ORDER BY c.relevanceScore DESC,
         c.uploadDate DESC
```

Requires a matching composite index:

```
[
  { "path": "/relevanceScore",
    "order": "descending" },
  { "path": "/uploadDate",
    "order": "descending" }
]
```

Filter + sort optimization

Include filter property in ORDER BY to use one composite index:

```
SELECT * FROM c
WHERE c.documentType = 'pdf'
ORDER BY c.documentType,
         c.uploadDate DESC
```

Composite index rules

- Property sequence must match ORDER BY exactly
- Supports reversed order on all paths
- Mixed orders need separate indexes
- Equality filters first, then one range filter last
- Each composite index adds write overhead

© Copyright Microsoft Corporation. All rights reserved.

Tune vector indexes for embedding workloads

Type	Best for	Max dims
flat	<1K vectors, exact	505
quantizedFlat	1K-50K vectors	4,096
diskANN	>50K vectors	4,096

Key decisions

- quantizedFlat and diskANN need 1,000+ vectors to activate
- Below threshold: falls back to brute-force scan
- DiskANN recommended for production AI apps
- Exclude embeddings from range indexes
- Vector policies are immutable after container creation
- float16 saves 50% storage vs float32
- Pre-filtering with metadata indexes reduces vector search RU cost

© Copyright Microsoft Corporation. All rights reserved.

Reduce RU costs with strategic indexing

Analyze before indexing

- Catalog filter properties (WHERE clauses)
- Catalog sort properties (ORDER BY)
- Identify property combinations in queries
- Prioritize by query frequency

Query metrics to monitor

- Index utilization percentage
- Retrieved vs. output document count
- RU charge per query (x-ms-request-charge)

Selective indexing strategy

- Exclude all by default: `"excludedPaths": [{"path": "/*"}]`
- Include only queried properties
- Don’t index: large text, summaries, raw metadata
- Always include partition key path
- Focus composite indexes on top 5-10 query patterns

Workload profile

- Write-heavy: minimize indexes, accept higher query cost
- Read-heavy: invest in indexes, most AI apps are here
- Each composite index increases write RU cost

© Copyright Microsoft Corporation. All rights reserved.

Choose consistency levels for optimal performance

Level	Replicas	RU cost
Strong	2	2x
Bounded staleness	2	2x
Session	1	1x
Consistent prefix	1	1x
Eventual	1	1x

Guidance for AI apps

- Session: most AI apps, read-your-writes guarantee
- Eventual: analytics, background processing, lowest cost
- Strong: financial or regulatory scenarios only, 2x RU cost
- Pass session tokens from writes to subsequent reads
- Create separate client instances per consistency level
- Vector search tolerates eventual consistency in most cases

© Copyright Microsoft Corporation. All rights reserved.

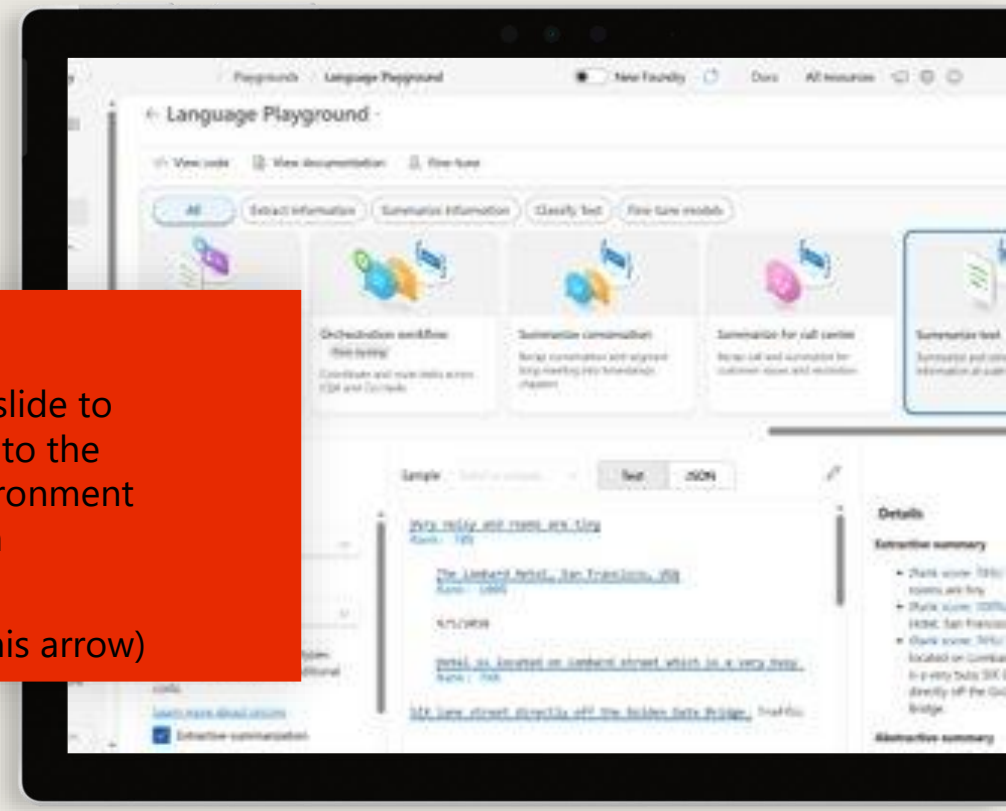
Exercise

Optimize query performance with vector indexes on Azure Cosmos DB for NoSQL

In this exercise, you implement vector similarity search using Azure Cosmos DB for NoSQL.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2350800



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 3 knowledge check

1

A developer needs to optimize a query that filters by documentType and sorts by uploadDate descending. Which index configuration best supports this pattern?

A. Create a composite index with documentType (ascending) followed by uploadDate (descending)
B. Create separate range indexes on documentType and uploadDate
C. Create a composite index with uploadDate (descending) followed by documentType (ascending)

2

Query metrics show low index utilization and high retrieved-to-output document ratios. What does this indicate?

A. The query is performing scans instead of using indexes. Add range or composite indexes for the query's filter and sort properties.
B. The container has too many indexes. Remove indexes to reduce query overhead.
C. The query returns too many results. Add a TOP clause to limit output

© Copyright Microsoft Corporation. All rights reserved.

Module 3 summary

- **Index types:** range for filters, composite for multi-property sort, vector for similarity, spatial for geo queries
- **Composite indexes** must match exact property sequence and sort direction in ORDER BY clauses
- **Vector index selection:** flat for small datasets, quantizedFlat for medium, diskANN for large-scale production
- **Selective indexing** excludes unused properties. Always exclude embedding arrays from range indexes
- **Query metrics** reveal missing indexes through low utilization and high retrieved-to-output ratios
- **Session consistency** provides read-your-writes for user-facing operations at 1x RU cost

© Copyright Microsoft Corporation. All rights reserved.

Group discussion

© Copyright Microsoft Corporation. All rights reserved.

Group discussion questions

- When using a document database like Azure Cosmos DB for NoSQL, what factors most influence how you model data for AI scenarios such as search, recommendations, or RAG?
- Cosmos DB offers a lot of flexibility, but also many design choices (partition keys, indexing, consistency). How do you decide when to optimize early versus keeping things simple and evolving later?
- AI models, embeddings, and query patterns change over time. How do you design data stores and architectures that can adapt without major rework or migrations?

© Copyright Microsoft Corporation. All rights reserved.

End of presentation

© Copyright Microsoft Corporation. All rights reserved.